

Building Stevie Social's all-in-one agency platform

A hybrid approach—custom frontend over battle-tested APIs and open-source tools—delivers the best balance of cost, speed, and flexibility for consolidating 14 tools into one unified system. The realistic investment ranges from \$75,000–\$150,000 over 12–18 months for a skilled solo developer, with monthly infrastructure costs of \$150–650.

However, a faster path exists: using GoHighLevel (\$497/month) as a foundation and building custom modules only for the gaps, cutting the custom build to \$30,000–\$60,000. The right strategy depends entirely on whether Stevie Social wants a tailored competitive advantage or the fastest time-to-value.

This report breaks down every dimension of the build—technology, costs, phasing, APIs, video handling, and AI—with specific recommendations a developer can act on immediately.

The three strategic paths and why hybrid wins

The build-vs-buy decision is the first and most consequential choice. Three viable strategies exist, each with fundamentally different tradeoffs.

Path A: GoHighLevel as foundation (\$30K–\$60K custom + \$497/month). GoHighLevel covers **10–11 of the 14 required functional areas** out of the box: website/funnel builder, calendar booking, invoicing via Stripe, email marketing, forms/surveys, automation workflows, social media scheduling, CRM, and AI features. KRON4 Its SaaS Pro plan at \$497/month offers full white-label capability— Top GHL Snapshots +2 Stevie Social could brand it entirely as their own. The gaps are real but contained: no robust project management (only basic tasks, no Kanban/Gantt/dependencies), no video recording or editing, limited large file storage (50MB upload cap), HighLevel basic social media analytics, and no dedicated proposal builder. Custom development would focus exclusively on filling these gaps—building a project management module, video asset management layer, and deeper analytics dashboard—at roughly **\$30,000–\$60,000** in development cost. This is the fastest path to market (8–12 weeks to deploy GHL + 3–6 months for custom extensions).

Path B: Full hybrid custom build (\$75K–\$150K). Build a custom Next.js + FastAPI frontend and business logic layer, but plug in proven services for commodity features: Stripe for payments, Cal.com for scheduling, Resend for email, Cloudflare R2 for storage, Mux for video streaming. This approach gives **complete control over UX, data, and functionality**

while avoiding the multi-year effort of building payment processing or email infrastructure from scratch. The developer builds roughly 6–8 custom modules while integrating 6–8 existing services. Timeline: 12–18 months solo, 9–15 months with a 2–3 person team. This is the recommended path if Stevie Social values differentiation, full data ownership, or plans to eventually resell the platform to other agencies.

Path C: Full custom build (\$150K–\$300K+). Building everything from scratch—including payment processing, email delivery infrastructure, and scheduling logic—is financially irrational for a single agency. The hybrid approach delivers identical end-user functionality at 40–60% lower cost by leveraging services that thousands of engineers have already hardened. The only justification for Path C is if Stevie Social plans to become a software company selling to hundreds of agencies, and even then, starting with the hybrid approach and internalizing services later is the smarter sequence.

The critical math on ROI: Stevie Social’s current 14-tool stack likely costs \$500–\$1,500/month. At \$12,000–\$18,000/year in subscriptions, a \$120,000 custom build takes **7–10 years** to break even on subscription savings alone. The ROI case must rest on operational efficiency gains, competitive differentiation, or—most compellingly—**reselling the platform to other agencies**. Twenty agency clients paying \$300/month = \$6,000/month revenue, achieving payback in under two years.

Recommended technology stack optimized for the developer’s skills

The developer’s strong Python, database, and automation background directly informs this stack recommendation. Every choice maximizes existing expertise while using best-in-class tools for each layer.

Frontend: Next.js 15 + Tailwind CSS + shadcn/ui. Next.js dominates SaaS development in 2025–2026 with server-side rendering for SEO DesignRevision (critical for the website/analytics module), API routes for lightweight backend logic, and the largest React component ecosystem. A fintech team switching to Next.js reported **40% faster deployments**. The shadcn/ui component library provides production-ready, accessible UI components that dramatically accelerate dashboard development. DesignRevision

Backend: FastAPI (Python) on Railway. FastAPI’s async performance rivals Node.js while keeping the entire Python ML/data ecosystem accessible—crucial for AI features and analytics processing. Pydantic type validation creates typed contracts across the stack when paired with TypeScript on the frontend. The architecture splits cleanly: Next.js handles SSR, static pages, and light API routes on Vercel; FastAPI handles core business logic,

heavy API endpoints, and data processing on Railway (\$5–20/month). Multiple sources explicitly call Next.js + FastAPI “the perfect stack for SaaS MVPs.”

Database: PostgreSQL via Supabase. PostgreSQL is the most recommended database for relational SaaS data—client records, projects, invoices, permissions—with JSONB support for flexible analytics data and form responses. Supabase wraps PostgreSQL with auto-generated REST APIs, built-in authentication, realtime subscriptions, storage, and edge functions. The **free tier includes 500MB database, 1GB storage, and 50K monthly active users**; the Pro plan is \$25/month. Row Level Security enforces multi-tenant data isolation at the database level, a critical security requirement.

Background processing: Celery + Redis. The developer’s Ignition/automation experience maps directly to task orchestration. Celery handles scheduled social media posts, report generation, email campaigns, video processing, and AI API calls. Redis serves double duty as Celery’s message broker and the platform’s caching layer. NightCoders

Authentication: Clerk or Supabase Auth. Clerk (\$0 for first 10K monthly active users) provides pre-built organization switching, member management, invitation flows, and role-based access that work out of the box—saving weeks compared to building from scratch. If budget is tight, Supabase Auth offers 50K free MAUs with integrated Row Level Security, though multi-tenancy requires custom implementation.

Component	Tool	Monthly cost (MVP)
Frontend hosting	Vercel	Free–\$20
Backend API	Railway (FastAPI)	\$5–20
Database + Auth + Realtime	Supabase	Free–\$25
Background workers	Railway (Celery)	Included
File/video storage	Cloudflare R2	\$15–75
Video streaming	Mux	\$20–50
Payments	Stripe	Transaction fees only
Email (transactional)	Resend	Free–\$20
Email (marketing)	Listmonk + Amazon SES	\$0–10
Scheduling	Cal.com (self-hosted)	\$0
Search	PostgreSQL FTS → Typesense	\$0–20

Every API and integration mapped to each functional area

Each of Stevie Social's 14 tool replacements requires specific APIs. Here is the precise integration map with pricing, capabilities, and gotchas.

Payments (replacing QuickBooks + Stripe): Stripe remains the backbone, but use its full product suite. **Stripe Billing** handles subscriptions and recurring invoicing (0.5% of recurring charges + standard 2.9% + \$0.30 per transaction). **Stripe Checkout** provides a pre-built, hosted payment page. **Stripe Customer Portal** lets clients self-manage subscriptions. **Stripe Invoicing** sends professional invoices (0.4% per paid invoice, first 25/month free). **Stripe Tax** automates sales tax calculation (0.5% per transaction). Total integration effort: approximately 2–3 weeks for a full billing system. This replaces QuickBooks for client billing entirely—for the agency's own bookkeeping, a QuickBooks API integration or simple CSV export may still be needed.

Calendar booking (replacing Calendly): Cal.com is the clear winner—an open-source Calendly alternative with **38,000+ GitHub stars**, full REST API, webhooks, round-robin scheduling, Stripe payment integration, and embeddable widgets. Self-hosting is free (Docker-based, Next.js + Prisma + PostgreSQL). Building custom booking logic from the Google Calendar API alone would require 3–6 months of development for availability logic, conflict detection, timezone handling, reminders, and embed functionality. Cal.com provides all of this immediately.

Email marketing (replacing Flodesk): A two-layer approach keeps costs near zero. **Resend** (\$20/month for 50K emails) handles transactional emails—password resets, notifications, booking confirmations—with excellent React Email template support matching the Next.js stack. **Listmonk** (open-source, self-hosted, PostgreSQL-based) handles marketing campaigns—newsletters, drip sequences, segmentation, analytics—paired with Amazon SES at \$0.10 per 1,000 emails [Dreamlit AI](#) for actual delivery. This combination costs roughly **\$2–20/month** versus Flodesk's \$35+/month, with no subscriber limits.

Forms and surveys (replacing Typeform): Build directly into the platform using **SurveyJS** (MIT-licensed) for maximum control—a JavaScript library providing a drag-and-drop form builder and renderer that works with React, stores form definitions as JSON in PostgreSQL, and handles conditional logic, multi-page forms, and file uploads. [SurveyJS](#) Alternative: **Formbricks** (open-source Typeform clone, 10K+ GitHub stars) if a more complete standalone solution is preferred. [House Of FOSS](#)

Automation workflows (replacing Zapier): **n8n** is the premier open-source automation

platform with **177,000+ GitHub stars** n8n and 400+ integrations. Self-host it and integrate via API to offer “Zapier-like” workflow building to Stevie Social’s team. Supports JavaScript and Python code nodes. Snoopforms For backend task processing (scheduled posts, report generation), use Celery as described above. The combination of n8n for visual workflows + Celery for backend jobs covers everything Zapier does.

Social media APIs (replacing Metricool) represent the most complex integration challenge:

Platform	Cost	Can publish?	Analytics depth	Key limitation
Meta (FB/IG)	Free	Yes (100 posts/day IG)	Deep (reach, engagement, demographics)	Complex permissions; frequent deprecations
TikTok	Free	Yes (requires partner status)	Moderate (views, likes, completion rates)	Approval required; 2+ week timeline
LinkedIn	Free	Yes (organic + paid)	Good (impressions, clicks, demographics)	Separate APIs for organic vs. paid
X/Twitter	\$200/month Basic	Yes (3,000 posts/month)	Limited at Basic tier	\$5,000 gap to Pro tier
Pinterest	Free	Yes	Good (impressions, saves, clicks)	Limited daily API calls for new apps
YouTube	Free	Yes (6 uploads/day default)	Excellent (watch time, retention, traffic)	10,000 quota units/day

Critical architectural decision: Rather than maintaining six separate API integrations (each with different OAuth flows, token lifetimes, and rate limits), consider a **unified social media API** like Ayrshare or the open-source **Postiz** project (TypeScript/NestJS, supports 10+ platforms with scheduling, analytics, and AI content generation). Postiz could serve as a reference architecture or starting codebase for the scheduling and publishing module.

Video handling: store cheaply, stream smartly, don’t build an editor

Video is Stevie Social's bread and butter, so this architecture must handle large files efficiently while keeping costs manageable.

Storage: Cloudflare R2 is the clear winner for a media-heavy agency. At \$0.015/GB/month with **zero egress fees**, [Create AI](#) storing and serving 5TB of video costs approximately \$75/month. The same on AWS S3 with typical egress patterns would cost \$500–1,000+/month. [nOps](#) [Substack](#) R2 is fully S3-compatible, meaning all existing S3 SDKs and tools work without modification. For archival storage of older assets, Backblaze B2 at \$0.006/GB/month (\$6/TB) paired with Cloudflare CDN [Blocks and Files](#) provides even cheaper cold storage with free egress through the Backblaze-Cloudflare alliance.

[Backblaze](#)

[Backblaze](#)

Streaming and playback: Mux provides the best developer experience for embedding video into a SaaS platform. Its just-in-time encoding only creates streaming versions when viewers actually watch (saving encoding costs), [Mux](#) includes built-in analytics (viewer engagement, quality of service), [Mux](#) and supports signed playback URLs for secure client sharing. Pricing starts at \$20/month (Launch plan with \$100 in usage credits). [Mux](#) For the agency's review/approval workflow—where team members and clients need to watch, comment on, and approve video content—Mux's API-first approach [Mux](#) with webhook events (upload complete, asset ready) integrates naturally into the project management pipeline.

Recording (replacing Loom): Browser-based screen recording is fully viable in 2025–2026 using the `getDisplayMedia()` + `MediaRecorder` Web APIs. [Web.dev](#) The open-source **RecordRTC** library handles cross-browser quirks and simplifies combining screen, camera, and audio streams. The flow: record in-browser → upload chunks to Cloudflare R2 → process via serverless function → serve via Mux for playback. This gives full platform control with no third-party dependency. Total implementation effort: approximately 2–3 weeks.

Editing (replacing Descript): Do not attempt to build a video editor. A full non-linear editor is a multi-year, multi-million-dollar effort (Descript has raised \$100M+). Instead, the custom platform should handle **organization and review**—uploading, tagging, searching, timestamped commenting, approval chains, and version tracking. For templated/automated video generation (creating branded social clips from templates), **Remotion** (React-based video creation) or **Shotstack** (cloud API, \$49/month) [Subscribed](#) can generate videos programmatically. For creative editing, Descript or CapCut remain external tools. The platform provides a “send to Descript” deep link and receives edited output back for the review/approval workflow.

File organization architecture: Store file metadata (client, campaign, platform target, resolution, duration, status, tags) in PostgreSQL alongside Cloudflare R2 object keys. Folder

structure follows: Client → Campaign → Asset Type (Raw/Edited/Final) → Date. Generate thumbnails on upload via Cloudflare Workers. Build search against database metadata, not the object store. This gives instant search and filtering across thousands of video assets.

AI features that enhance without replacing the human touch

Stevie Social’s concern about maintaining personality is valid and architecturally addressable. The key principle: **AI generates drafts and suggestions; humans make decisions and add voice.**

Multi-model strategy for cost and quality optimization. Use **Claude Sonnet** (\$3/million input tokens, \$15/million output tokens) [PoloAPI](#) for creative content generation—it consistently produces more natural, less “robotic” marketing copy than GPT models. Use **GPT-4o mini** (\$0.15/million input tokens, \$0.60/million output tokens) [Cloudidr](#) for utility tasks—quality checks, sentiment analysis, task suggestions, structured data extraction. An abstraction layer (using LiteLLM or Vercel AI SDK) lets you swap models per feature without code changes. [Unified AI Hub](#)

Estimated AI API costs for 50–100 content pieces/month:

Task	Model	Monthly cost
Content drafts (captions, posts)	Claude Sonnet	\$15–30
Quality checks (grammar, brand compliance)	GPT-4o mini	\$1–3
Analytics insights reports	Claude Sonnet	\$5–10
Task/workflow suggestions	GPT-4o mini	\$1–2
Comment sentiment analysis	GPT-4o mini	\$2–5
Total		\$25–50/month

At this volume, self-hosting open-source models (Llama, Mistral) makes no economic sense—break-even requires thousands of daily requests. API costs are negligible.

Per-client brand voice profiles solve the “human touch” problem technically. Store each client’s tone descriptors, vocabulary preferences, example content, audience details, and “do not” lists in the database. Inject these dynamically into system prompts for every AI call for that client. Include 3–5 examples of ideal content (few-shot prompting) to maintain consistency. Let users rate AI outputs with thumbs up/down, then use top-rated outputs as

future few-shot examples—creating a continuous improvement loop.

Architecture pattern for AI integration: All AI operations must be **async by default**.

[Ctco](#) User action → API gateway (rate limiting, auth) → job queue (Redis/Celery) → AI worker service → provider abstraction layer → OpenAI/Claude → webhook/SSE to update UI. Never block the UI waiting for AI responses. Implement per-user and per-organization rate limits and hard monthly spending caps to prevent runaway costs. [AppZoro](#) Use prompt caching (both OpenAI and Anthropic offer up to 90% savings on cached inputs) and batch APIs (50% discount for non-urgent processing) to further reduce costs. [Solvimon](#)

Seven-phase development roadmap with timelines and costs

Each phase delivers working functionality Stevie Social can use immediately, building on previous phases while maintaining independence where possible.

Phase 1: Website + analytics + calendar booking (replacing Squarespace + Calendly).

Build a custom marketing website with Next.js, embed Plausible or Umami for privacy-friendly visitor analytics (self-hosted, free), and integrate Cal.com for booking. This is the fastest win—a working replacement for two tools that demonstrates immediate value. *Solo developer: 8–12 weeks, \$12,000–\$20,000. Team: 4–6 weeks, \$15,000–\$25,000.* Technical challenges: SEO performance matching Squarespace, timezone handling for calendar, real-time analytics pipeline. Dependencies: none.

Phase 2: Financial management (replacing QuickBooks + Stripe). Stripe Billing for subscriptions, Stripe Invoicing for one-off invoices, Stripe Customer Portal for self-service, custom dashboard for revenue tracking. PDF invoice generation. Basic expense categorization. *Solo: 10–14 weeks, \$15,000–\$25,000. Team: 5–8 weeks, \$18,000–\$30,000.* Technical challenge: accounting accuracy and PCI compliance (Stripe handles PCI). Dependency: Phase 1 (shared auth/user system).

Phase 3: Proposals + client onboarding. Template-based proposal builder with drag-and-drop editing. E-signature integration (HelloSign API, ~\$1,500 to integrate). Auto-conversion of accepted proposals into projects. Automated onboarding sequences—welcome emails, intake forms, file collection checklists, client portal access provisioning. *Solo: 6–10 weeks, \$10,000–\$18,000. Team: 3–5 weeks, \$12,000–\$20,000.* Dependency: Phase 2 (billing triggers after proposal acceptance).

Phase 4: Project management (replacing ClickUp). Kanban boards mapping to Stevie Social's 13-step workflow, task assignment, due dates, priorities, file attachments, comments/activity feeds, time tracking, and client-facing project views. Focus on the **20% of ClickUp features that cover 80% of agency use**—boards, lists, basic views, and the

specific approval workflow for Step 9 (Client Approval & Revisions). *Solo: 10–16 weeks, \$15,000–\$28,000. Team: 6–9 weeks, \$20,000–\$35,000.* This is the most complex module. Technical challenges: real-time collaboration via WebSockets, complex drag-and-drop UI, performance with large datasets.

Phase 5: Reporting and analytics dashboard. Client performance dashboards aggregating data from all previous modules. Revenue reports, project status reports, customizable report builder, PDF export, scheduled email delivery. Chart rendering with Recharts or Chart.js. *Solo: 6–10 weeks, \$8,000–\$15,000. Team: 3–5 weeks, \$10,000–\$18,000.* Dependency: needs data from Phases 1–4.

Phase 6: Email marketing + forms + automation (replacing Flodesk + Typeform + Zapier). Listmonk integration for campaign management, SurveyJS for form building, n8n for visual workflow automation. Drag-and-drop email template builder (the hardest UI piece in this phase). Contact segmentation. *Solo: 12–18 weeks, \$18,000–\$30,000. Team: 7–10 weeks, \$22,000–\$38,000.* Email deliverability is one of the hardest problems in SaaS—using SendGrid/Resend as the delivery layer rather than building SMTP infrastructure is non-negotiable.

Phase 7: Video + social analytics + AI (replacing Dropbox + Descript + Loom + Metricool + ChatGPT). Cloudflare R2 storage with metadata management, Mux streaming integration, browser-based recording via RecordRTC, social media API integrations (Meta, TikTok, LinkedIn, Pinterest, YouTube), AI content generation with brand voice profiles. *Solo: 12–18 weeks, \$18,000–\$30,000. Team: 7–10 weeks, \$22,000–\$38,000.*

Total for hybrid build: Solo developer: **\$96,000–\$166,000 over 18–28 months** (add 25% contingency: **\$120,000–\$208,000**). A 2–3 person team: **\$119,000–\$224,000 over 9–15 months** (with contingency: **\$149,000–\$280,000**). AI-assisted development tools (Cursor, Claude, Copilot) can realistically reduce these estimates by 20–30%, as a skilled solo developer in 2026 can match the output of a 2–3 person team from 2023.

How to scope and price this for the client

Start with a paid discovery phase. Charge \$2,000–\$5,000 for a 2–3 week deep-dive producing a detailed technical specification, architecture document, wireframes for key screens, and refined cost estimate. This protects both parties and demonstrates professionalism.

Phase 1 as a fixed-price proof of concept. Quote Phase 1 (website + analytics + booking) at a fixed \$12,000–\$18,000 with a clear deliverable and timeline. This builds trust, delivers tangible value [CodeSuite](#) in 8–12 weeks, and gives both sides data on working velocity

before committing to the full build.

Subsequent phases: milestone-based pricing with $\pm 15\%$ variance allowance per phase. Quote each phase separately, allowing scope refinement between phases. This is the lowest-risk model for both developer and client.

Post-launch: \$3,000–\$5,000/month retainer covering maintenance (bug fixes, security patches, dependency updates, API version changes), hosting costs, and ongoing feature development. Industry standard maintenance runs 15–20% of initial build cost annually.

Luzmo

The opportunity reframe: If Stevie Social is interested in the platform becoming a product sold to other agencies, negotiate an equity stake or revenue-share component alongside the development fee. A multi-tenant version serving 20 agencies at \$300/month generates \$72,000/year in recurring revenue—fundamentally changing the economics from a cost center to a revenue stream.

The strongest opening recommendation for the client: **start with GoHighLevel at \$497/month to validate the all-in-one concept immediately**, while beginning Phase 1 custom development in parallel. This gives Stevie Social an operational platform within weeks while the custom solution is built to eventually replace it—and the GHL experience reveals which custom features matter most, reducing waste in the custom build.

Conclusion

The 14-tool consolidation Stevie Social envisions is technically achievable and architecturally sound. The hybrid approach—custom frontend, proven API services for commodity features, strategic open-source tools—delivers the optimal balance. The developer's Python expertise maps directly to the recommended FastAPI + Celery + PostgreSQL backend. **Cloudflare R2's zero-egress pricing** [Create AI](#) [Cloudflare](#) is transformative for a video-heavy agency. AI integration costs are negligible at \$25–75/month for the expected volume.

The critical insight is that this project's value scales nonlinearly if built as multi-tenant SaaS from day one. A platform serving only Stevie Social is an expensive custom tool with a 7–10 year payback. A platform serving 20 agencies is a SaaS business with 2-year payback and growing recurring revenue. The architecture recommendations above support both scenarios—the multi-tenant patterns (Row Level Security, organization switching, white-labeling) add roughly 15% to initial development cost but unlock an entirely different business model.

The recommended first step: a paid discovery sprint, followed immediately by Phase 1

development and a parallel GoHighLevel deployment for operational continuity.